

Eötvös Loránd University

Faculty of Informatics

Department of Programming Theory and Software Technology

Interactive Application of Search Algorithms and Visualization of State Graphs

Supervisor:

Dr. Tibor Gregorics

Associate Professor, Head of Department

Author:

Szabolcs Ádám Szécsi

BSc in Computer Science

Budapest, 2026

Contents

1 Introduction **3**

2 User Documentation **5**

2.1 Problem Definition 5
2.2 Overview of the Methods 6
2.3 System Requirements 7
2.4 Application Overview 8
2.5 Loading a Level 9
2.6 Manual Play 9
2.7 Configuring the Search Algorithm 10
2.8 Running Search Algorithms 11
2.9 Discovering States 12
2.10 Interacting with the 3D State Graph 12
2.11 Color Legend 13

3 Developer Documentation **15**

3.1 Analysis 15
3.1.1 Task Description 15
3.1.2 Functional Description 16
3.1.3 Nonfunctional Requirements 19
3.1.4 Related Work 19
3.2 Design 19
3.2.1 Software Architecture 19
3.2.2 Model 20
3.2.3 View 22
3.2.4 Data Storage 25
3.2.5 Search Algorithms and Heuristics 25
3.3 Implementation 30

3.3.1 Technologies Used 30
3.3.2 Deviations from the Original Design 30
3.4 Testing 31
3.4.1 Unit Tests 31
3.4.2 End-User Test Cases 35

4 Conclusion **39**

Bibliography **40**

List of Figures **42**

List of Tables **43**

Chapter 1

Introduction

Search algorithms play a central role in computer science, particularly in artificial intelligence [1]. These algorithms are essential for pathfinding, game analysis, and automated reasoning. Although their theoretical properties—such as completeness, optimality, and computational complexity—are well understood, few interactive tools are available to help learners understand how they work.

Visualizing the explored state space can make the rules governing a search algorithm's decisions much clearer. A visualization of its global working memory reveals the knowledge the algorithm has acquired, allows the role of heuristic functions to be analyzed, and makes it possible to compare the results produced by different configurations.

This thesis presents an interactive application that applies search algorithms to the Rush Hour logic puzzle [2] and visualizes the explored state graph in three-dimensional space. Rush Hour is a sliding-block puzzle in which the player moves vehicles on a 6×6 board to clear a path to the exit for the main (red) car. Although the rules are simple, the puzzles range from easy to surprisingly complex—the generalized version has been shown to be PSPACE-complete [3]—making it an ideal problem for this analysis.

The project approaches the problem from two directions:

1. **Implementation and control of search algorithms.** Three search algorithms have been implemented—tabu search, backtracking search, and A^C graph search. Each can be configured with one of four heuristic functions and an optional randomization setting. Users can also play the puzzle manually, and every discovered state appears in the state graph.

2. **Three-dimensional state graph visualization.** Discovered states appear as circular vertices in three-dimensional space, connected by edges representing valid moves. A force-directed layout [4] uses a physical simulation to move the vertices so that neighboring states remain close together while non-neighboring states move apart. To address the $O(V^2)$ cost of computing repulsive forces, the Barnes–Hut algorithm [5] and an octree spatial data structure are used, reducing the complexity to $O(V \log V)$ per frame, where V is the number of discovered vertices.

The application was developed using Godot Engine 4.6 [6], C#, and .NET 9.0 [7], and follows the Model–View architecture [8]. The model layer encapsulates the puzzle logic, search algorithms, and heuristic functions in plain C# constructs. The view layer implements the physical simulation, rendering, and user interaction using Godot's scene system.

Chapter 2

User Documentation

This chapter provides a complete description of the application for users. It begins with a summary of the problem, introduces the methods used, and then explains how to run and interact with the program.

2.1 Problem Definition

Rush Hour [2] is a sliding-block puzzle played on a 6×6 square grid. The board can contain two types of vehicles: **cars**, occupying 1×2 cells, and **buses**, occupying 1×3 cells. Each vehicle can move only forward and backward along its longer axis, and vehicles cannot push one another. The objective is to slide the designated red **main car** horizontally until it reaches the exit on the right edge of the board in the third row.

The puzzle is considered **solved** when the front of the main car is immediately next to the exit. The game's state space consists of all configurations reachable from the initial state. Transitions between states correspond to moving a single vehicle by one cell in a permitted direction.

The application focuses on two related challenges:

1. Implementing search algorithms for Rush Hour and providing interactive control over them, allowing users to pause, step through, and resume execution at will while analyzing each decision.

2. Visualizing the state graph in a three-dimensional scene, where vertices represent discovered states and edges represent permitted moves. A force-directed layout brings structurally related states close together.

2.2 Overview of the Methods

The application implements three search algorithms, each using a heuristic control strategy with configurable heuristic functions:

Tabu search A local search method with an irrevocable strategy: it selects the best neighboring state according to a heuristic while remembering recently visited states in a *tabu list* to avoid cycles.

Backtracking search A search method with a revisable strategy: it explores states in a heuristic-guided order, avoids cycles, and backtracks at dead ends. It is guaranteed to terminate on a finite state graph.

A^C graph search A graph search method with a revisable strategy, using the evaluation function $f(n) = g(n) + h(n)$, where $g(n)$ is the depth (the number of moves from the initial state) and $h(n)$ is the heuristic value. With admissible and monotone heuristics, this algorithm finds an optimal solution.

Four heuristic functions are available, all of which are nonnegative, admissible, and monotone: [9, pp. 14–15]

- **Null heuristic** ($h = 0$): provides no guidance and is useful for observing "brute-force" behavior.
- **Distance heuristic**: the distance of the main car from the exit (range: 0–4).
- **Distance + blocking heuristic**: the distance plus a cost of one for each occupied cell between the main car and the exit.
- **Least-moves heuristic**: provides a stronger lower bound on the moves needed to solve the puzzle: the distance plus the minimum number of moves required to move the vehicles blocking the main car's path out of the way.

Admissibility means that a heuristic never overestimates the actual minimum cost. Monotonicity means that a single move can change the heuristic value by at most 1. These properties ensure that A^C graph search finds an optimal solution. [9, p. 16]

A force-directed approach is used to arrange the 3D state graph: edges act as springs that pull neighboring states toward a given separation, while every pair of vertices exerts a repulsive force inversely proportional to the square of their distance. The computationally expensive $O(V^2)$ repulsion calculation is approximated using

the Barnes–Hut algorithm [5], implemented with an octree spatial data structure, reducing its time complexity to $O(V \log V)$.

2.3 System Requirements

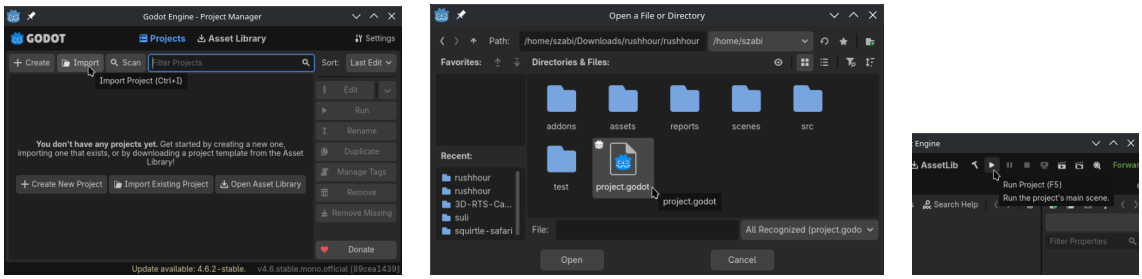
The application is available as an executable for Windows and Linux and requires no additional software installation. The project can also be run from the Godot editor and exported as an executable for other platforms. The recommended minimum screen resolution is 1280×720. The executables are available in the thesis project's GitHub repository.

Platform	How to run
Windows	Download and extract `windows_build.zip`, then run `rushhour.exe` from the extracted files.
Linux	Download and extract `linux_build.zip`, then run `rushhour.x86_64` from the extracted files.
macOS / other	Clone or download the repository, import the extracted `rushhour/project.godot` file into the Godot 4.6 Mono editor using Import Project, and run it using the button in the upper-right corner.

Table 2.1: Supported platforms and instructions for running the application

Prerequisites for opening the project in the editor:

- Download the Godot 4.6 Mono (C#) editor. [6]
- Install .NET SDK 9.0 or later. [7]

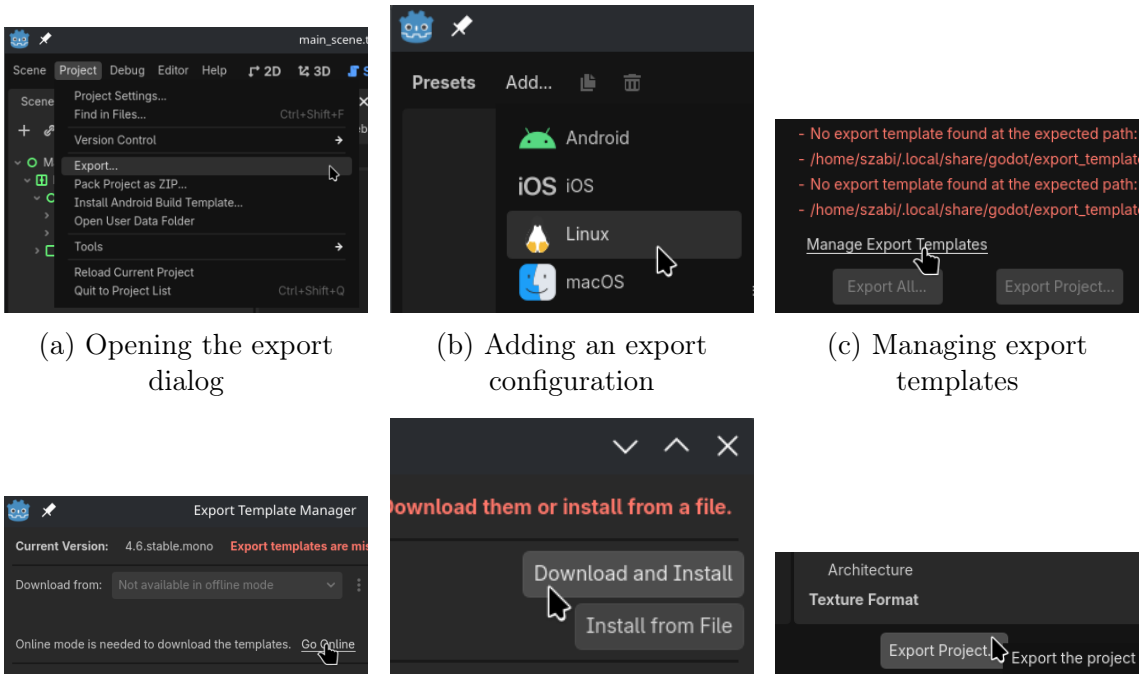


(a) Importing the project (b) Opening the project (c) Running the project

Figure 2.1: Opening and running the project from the Godot editor

To build from source once the prerequisites have been met, open the project and select Project > Export in the upper-left corner. Add an export configuration for

the chosen platform, then download the export templates when prompted by the editor.



(a) Opening the export dialog

(b) Adding an export configuration

(c) Managing export templates

(d) Going online

(e) Downloading export templates

(f) Exporting the project

Figure 2.2: Steps for exporting the project from the Godot editor

2.4 Application Overview

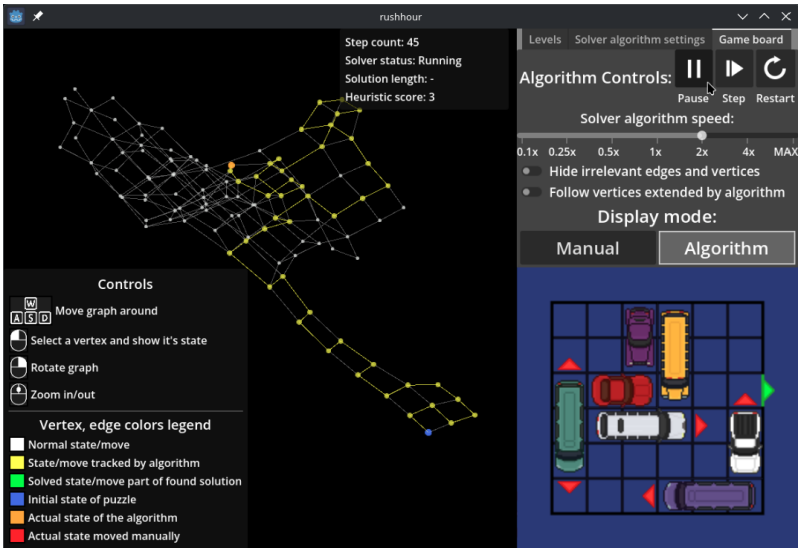


Figure 2.3: The application while an algorithm is running

The application window is divided into two main areas:

- **Left panel**: the scene displaying the 3D state graph.
- **Right panel**: configuration and control options, divided into three tabs.

The left panel contains:

- In the lower-left corner, the controls for interacting with the state graph and the color legend.
- In the upper-right corner, information about the currently running algorithm:
  - The number of steps performed by the algorithm.
  - The algorithm's current status.
  - The number of moves in the solution found.
  - The heuristic value of the state currently displayed on the board.

The right panel is divided into the following tabs:

- **Level selection tab**: select a puzzle by clicking one of the listed previews.
- **Solver settings tab**: start state discovery and choose the search algorithm and heuristic to apply to the selected puzzle.
- **Game board tab**: contains the game board, buttons for selecting Manual or Algorithm mode, algorithm controls for pausing, resuming, stepping, and restarting, a slider for the algorithm's execution speed, and switches controlling the state graph's appearance.

2.5 Loading a Level

The application contains a curated collection of Rush Hour puzzles of varying difficulty. Scroll through the list to view all puzzles.

A different puzzle can be selected at any time:

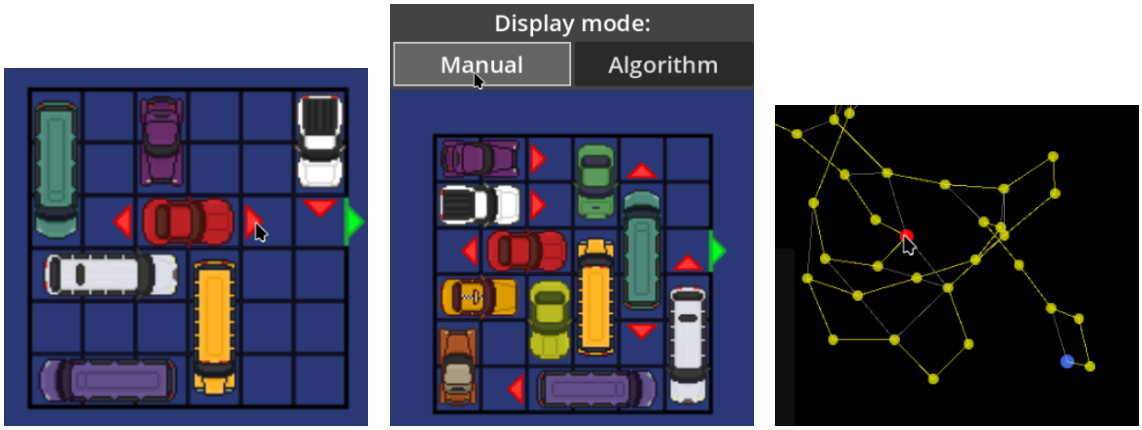
1. Open the **Levels** tab in the upper-right corner.
2. Click the button corresponding to the desired puzzle.
3. The game board loads the initial configuration, and a blue vertex representing the initial state appears in the 3D state graph.

2.6 Manual Play

The puzzle can be played manually by interacting directly with the game board:

1. Red arrow indicators appear next to each movable vehicle, showing the available movement directions.
2. Click an arrow to move the corresponding vehicle by one cell.
3. Each manual move creates a new state and its corresponding vertex and edge in the 3D graph, if the state has not already been discovered. This allows users to build the state graph interactively.

You can switch to manual mode at any time by pressing the **Manual** button or clicking the vertex of an already discovered state. In both cases, the board displays the selected state and switches to manual mode. If a search algorithm is currently running, it continues in the background. The vertex of the manually selected state is shown in red in the state graph.



(a) Solving the puzzle with manual moves, without a search algorithm

(b) The board stops following the algorithm's steps after switching to manual mode.

(c) Selecting a state by clicking a discovered vertex

Figure 2.4: Manual play and vertex selection

2.7 Configuring the Search Algorithm

Before running a search algorithm, users can configure its parameters on the **Solver settings** tab:

Algorithm selection Choose tabu search, backtracking search, or A^C graph search.

Tabu size (Tabu search only.) An integer $n \geq 0$ specifying how many recently visited states the system remembers.

- $n = 0$: greedy algorithm—always moves to the best neighbor without retaining any memory.
- $n = 1$: hill-climbing algorithm—remembers only the previous state and avoids returning to it.
- $n > 1$: avoids the last n visited states. If every possible move leads to a tabu state, the algorithm terminates without a solution.

Heuristic function Choose one of the four available heuristics: Null, Distance, Distance + Blocking, or Least-Moves Estimate.

Randomization switch When enabled, the algorithm chooses randomly among states with equal heuristic scores instead of selecting deterministically. This can produce different traversal paths on repeated runs.

Apply button Press this button to initialize the search algorithm for the loaded puzzle with the selected settings.

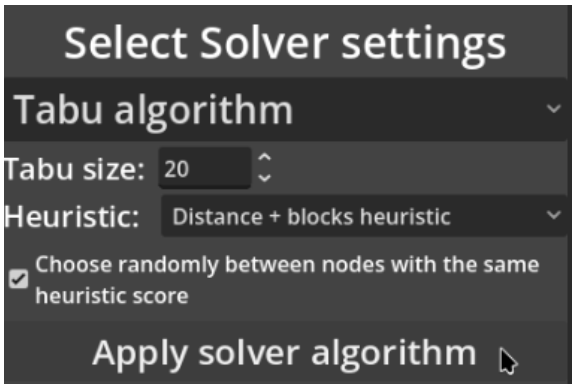


Figure 2.5: Configuring and applying search algorithms

2.8 Running Search Algorithms

After loading a level and configuring and applying the solver with the Apply button, the algorithm can be controlled from the **Game board** tab:

Play/Pause Start or pause continuous execution of the algorithm.

Step Execute a single step of the algorithm, which is useful for detailed observation.

Restart Reset the algorithm to the initial state while retaining the current settings.

Speed slider Set the delay between algorithm steps. Moving the slider to the right increases the speed. At its maximum setting, the algorithm runs as fast as possible and may execute hundreds of steps per frame.

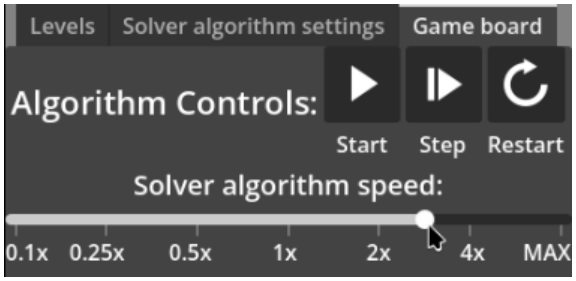


Figure 2.6: Controlling algorithm execution

When the algorithm finds a solution, the solution path is highlighted in green in the state graph, and the solution length is displayed.

2.9 Discovering States

In addition to the search algorithms, the application offers **BFS** (breadth-first search) and **DFS** (depth-first search) discovery modes, available from the Solver settings tab. These modes explore the state space without searching for a solution, adding every reachable state and edge to the state graph:

- A **maximum state count** can be set to limit the number of discovered states. This is recommended for more difficult puzzles, whose state spaces may contain thousands of elements: too many vertices can make the state graph difficult to interpret and slow down the physical simulation.
- The **unlimited** switch removes the state limit.

Discovery runs at high speed and populates the graph. Users can then run a search algorithm on the fully or partially explored state space. Discovery can also be controlled using the Play/Pause and Step buttons and the speed slider.

2.10 Interacting with the 3D State Graph

The application displays the state graph in the 3D area on the left. The following interactions are available:

Rotate the camera Hold down the right mouse button and drag the mouse to orbit around the graph.

Move the camera Use the `W`, `A`, `S`, and `D` keys to move the camera relative to its current orientation. Hold `Shift` to move faster.

Zoom Use the mouse wheel to zoom in and out.

Click a vertex Left-click a vertex to jump to that state: the game board switches to Manual mode and displays the selected state.

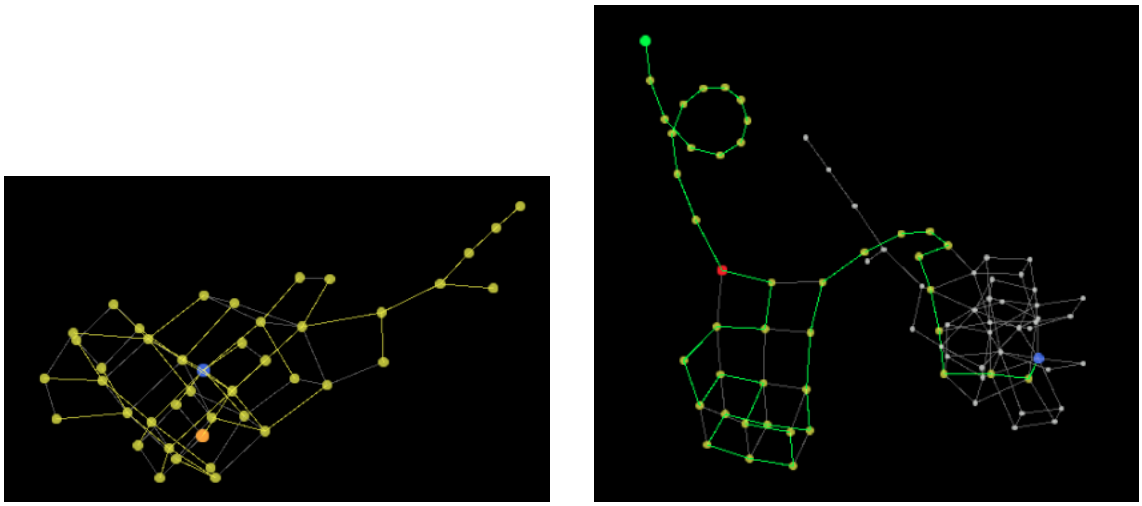
Hide vertices/edges A toggle on the Game board tab hides all vertices and edges that are not highlighted. This can reduce visual clutter when exploring large graphs.

2.11 Color Legend

The vertices and edges of the 3D graph are color-coded to indicate their roles:

Element	**Color**	**Meaning**
Vertex	Red	Current manually selected state
Vertex	Green	Solved state (main car at the exit)
Vertex	Blue	Initial state of the current puzzle
Vertex	Orange	Algorithm's current state
Vertex	Yellow	State stored in the algorithm's memory
Vertex	White	Discovered state
Edge	Green	Part of the solution path found
Edge	Yellow	Move stored in the algorithm's memory
Edge	White	Discovered move

Table 2.2: Color legend for the vertices and edges of the 3D state graph



(a) The spanning tree stored in the graph search algorithm's memory is shown in yellow; the current state is orange.

(b) A solved state and the edges of its solution path are highlighted in green. A selected state on the solution path is shown in red.

Figure 2.7: Illustration of the state graph's color coding

A state's vertex can have several properties at once: for example, it may be both selected and stored in the algorithm's memory. In such cases, the first applicable property in the table takes priority when determining its appearance. The first four properties (manual current state, solved state, initial state, and algorithm current state) are indicated by larger vertices as well as distinct colors. Vertices stored in the algorithm's memory are also slightly larger.

Chapter 3

Developer Documentation

This chapter presents the analysis, design, implementation, and testing of the application.

3.1 Analysis

3.1.1 Task Description

The task is to develop an interactive application that:

1. Models the Rush Hour sliding-block puzzle [2] as an immutable state-transition system on a 6×6 board with 1×2 cars and 1×3 buses.
2. Implements three configurable search algorithms (tabu search, backtracking search, and A^C graph search) with four heuristic functions.
3. Visualizes the explored state graph in 3D using a force-directed layout and Barnes–Hut optimization [5].
4. Supports manual play and interactive inspection of states by clicking graph vertices.

The generalized Rush Hour puzzle, played on an arbitrary $n \times n$ board, has been shown to be PSPACE-complete [3]. Hearn and Demaine [10] later introduced a more general framework establishing the same result for a broad class of sliding-block puzzles. Although the standard 6×6 board is finite and manageable in size, the state space can grow exponentially with the number of vehicles, justifying the use of efficient heuristics and search algorithms.

3.1.2 Functional Description

The functional requirements are summarized by the user stories in Table 3.1.

User Stories			
Goal	**Precondition**	**Event**	**Result**
Select a level	The level selection tab is open	I click the desired level's preview	The initial state is loaded; only the initial state appears in the graph; the right panel switches to the Solver settings tab
Discover state graph vertices	The Solver settings tab is open	I click the state discovery button	Discovered vertices are added to the state graph, with goal vertices highlighted
Pause state discovery	State discovery is running	I click the button to pause state discovery	State discovery pauses
Select a search algorithm	The Solver settings tab is open	I select the desired search algorithm from the options	The search algorithm is selected
Select a heuristic function	The Solver settings tab is open	I select the desired heuristic function from the options	The heuristic is selected
Apply the configured search algorithm	The Solver settings tab is open	I click the button to apply the search algorithm	The search algorithm is applied to the loaded puzzle with the selected settings

Goal	Precondition	Event	Result
Start or resume the search algorithm	The Game board tab is open	I click the button to start or resume the search algorithm	The algorithm starts; vertices and edges in its global working memory are highlighted; it stops if it finds a goal state
Pause the search algorithm	The Game board tab is open and a search algorithm is running	I click the button to pause the search algorithm	The algorithm pauses; vertices and edges in its global working memory remain highlighted
Run the search algorithm step by step	The Game board tab is open	I click the button to execute a single algorithm step	The algorithm executes one step; vertices and edges in its global working memory are highlighted
Set the search algorithm's speed	The Game board tab is open	I move the speed slider to the desired position	Each algorithm step is performed at the selected speed
Make a manual move on the board	The Game board tab is open	I click the red arrow corresponding to the desired move	The car moves on the board; the new state's vertex is discovered and highlighted in the graph
Switch the game board to manual mode	The Game board tab is open	I click the Manual button above the board	The board switches to manual mode; the displayed state is highlighted in the graph; any running algorithm continues in the background

Goal	Precondition	Event	Result
Display the algorithm's steps on the board	The Game board tab is open	I click the Algorithm button above the board	The board switches to Algorithm mode and displays the algorithm's most recently expanded state, then follows subsequent expansions
Jump to a discovered state	A puzzle is loaded	I left-click (M1) a discovered state's vertex	The board displays the selected state, and its vertex is highlighted in the graph
Rotate the state graph	A puzzle is loaded	I right-click (M2) the graph and drag the mouse	The graph rotates around the active vertex according to the mouse movement
Move the state graph	A puzzle is loaded	I press one of the WASD keys	The graph moves on the canvas according to the key pressed
Hide or show irrelevant edges and vertices	The Game board tab is open	I click the toggle for hiding irrelevant edges and vertices	Irrelevant edges and vertices are hidden or shown in the state graph
Follow vertices expanded by the algorithm	The Game board tab is open	I click the toggle for following the algorithm's vertices	The state graph camera follows the vertices expanded by the algorithm
Zoom in or out on the state graph	A puzzle is loaded	I use the mouse wheel over the state graph	The graph is zoomed in or out according to the wheel movement

Goal	Precondition	Event	Result
Exit the program	The program is running	I close the program window	The program closes

Table 3.1: User stories

3.1.3 Nonfunctional Requirements

- **Performance**: the 3D graph must remain interactive even with thousands of vertices. Barnes–Hut optimization must keep the computational cost per frame manageable.
- **Portability**: standalone executables for Windows and Linux, with support for building for other platforms through the Godot editor.
- **Usability**: an intuitive user interface with color-coded graph elements and responsive controls.
- **Correctness**: the search algorithms must exhibit their expected properties; for example, A^C graph search must find an optimal solution when using a monotone heuristic.

3.1.4 Related Work

Visualizing state-space search is not a new idea in the context of sliding-block puzzles. The 2swap YouTube channel [11] presents the Japanese Klotski puzzle, which is similar to Rush Hour. The graph structure shown there clearly demonstrates how visual tools can reveal the combinatorial structure of puzzles.

The present application additionally allows different search algorithms to be run on the puzzles, enabling real-time comparisons of heuristics and algorithms.

3.2 Design

3.2.1 Software Architecture

The application follows the **Model–View** architecture [8]:

Model layer (`src/Model/`) Contains plain C# classes for representing game states, generating moves, evaluating heuristics, and implementing search algorithms. The model is independent of the Godot engine except for its use of Godot's `Vector2I` structure, which avoids subsequent conversions.

View layer (`src/Nodes/`) Contains Godot node scripts derived from the engine's base classes, such as `Control`, `RigidBody3D`, and `MeshInstance3D`. These classes handle rendering, the physical simulation, the user interface, and input. They subscribe to model events (`NewEdge`, `PathChange`, `NewCurrent`, and `Terminated`) to respond to the algorithm's progress.

Communication between the model and the view is implemented through C# events defined on the abstract `RHSolver` base class. The view subscribes to these events and creates or updates the corresponding visual elements: vertices, edges, and highlights.

3.2.2 Model

Class diagrams of the main parts of the model are shown in Figures 3.1 and 3.2.

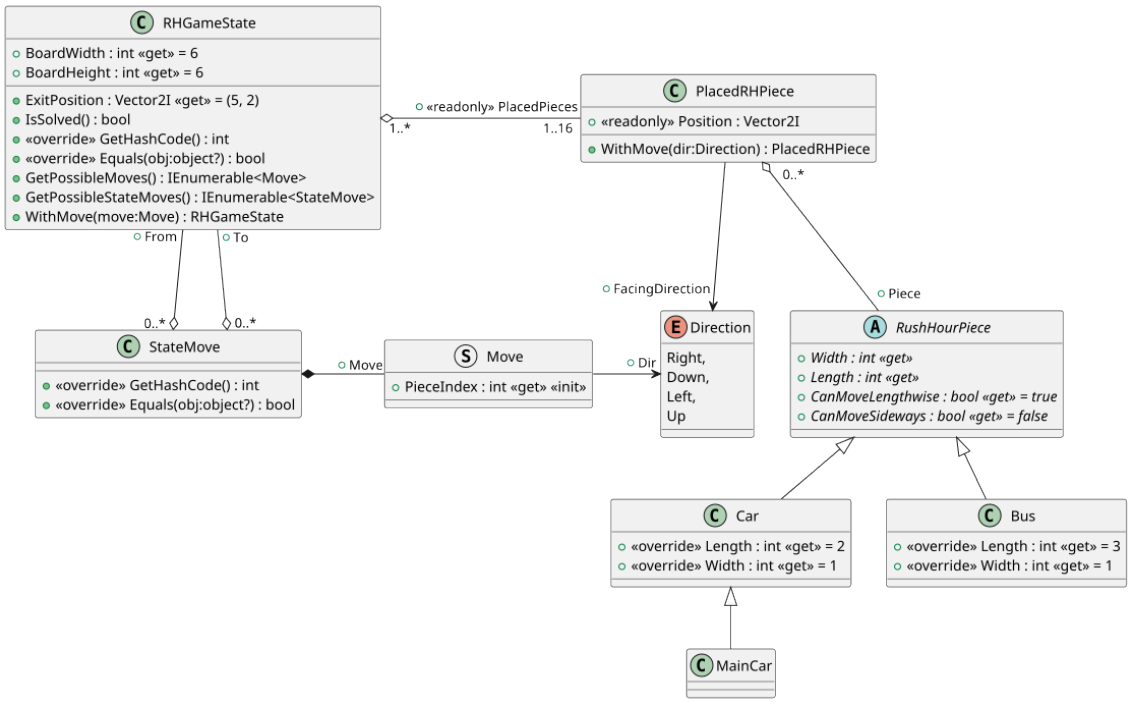


Figure 3.1: Class diagram of the state representation

States and pieces are all represented by immutable classes.

`RHGameState` An immutable representation of a board configuration. Immutability is ensured by the `init;` keyword and *readonly* data structures. It stores placed vehicles as an array of `PlacedRHPiece` objects and generates a 6×6 integer matrix from them for fast cell access. It provides the methods `GetPossibleMoves()`, `WithMove()`, and `IsSolved()`, as well as value-based equality checks accelerated by hash codes. Since the data structure is immutable, the hash code only needs to be calculated once during initialization and can then be returned on every subsequent request.

`PlacedRHPiece` Associates a `RushHourPiece` (Car, MainCar, or Bus) with a position and orientation on the board. Its `WithMove()` method returns a new instance with an updated position.

`Move` / `StateMove` A `Move` is a structure carrying little information on its own: a vehicle index and a direction. A `StateMove` records a complete transition: the source state, the target state, and the move itself. Since every move in the model can be reversed, equality checks do not distinguish a move from its reverse: $(A \to B) = (B \to A)$. The model therefore forms an undirected graph.

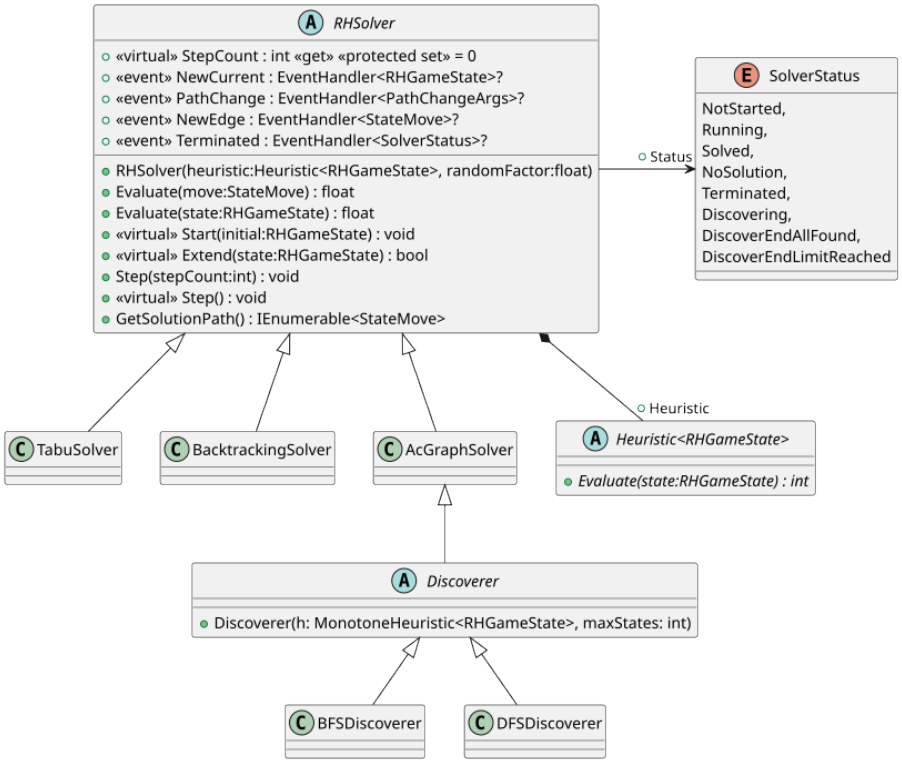


Figure 3.2: Class diagram of the search algorithms

`RHSolver` **hierarchy** The abstract `RHSolver` base class defines the event-driven interface and shared logic. Its concrete subclasses are:

- `TabuSolver`: maintains a list of recent states and selects the best non-tabu neighbor.
- `BacktrackingSolver`: maintains a stack of ordered option lists and backtracks at dead ends.
- `AcGraphSolver`: uses a `PriorityQueue` ordered by $f = g + h$ and tracks discovered states in a dictionary.
- `BFSDiscoverer` / `DFSDiscoverer`: the `Discoverer` class is based on graph search and therefore inherits from `AcGraphSolver`. It does not check for a solution and does not terminate when a solved state is expanded. Its breadth-first and depth-first subclasses override the evaluation function.

`Heuristic` **hierarchy** An abstract class hierarchy that enforces heuristic properties through the type system: `Heuristic` $\supset$ `NonNegativeHeuristic` $\supset$ `AdmissibleHeuristic` $\supset$ `MonotoneHeuristic`. The A^C graph search implementation requires a `MonotoneHeuristic` at compile time.

The model also includes the static `Levels` class, which contains strings describing the levels and a parser that converts them into `RHGameState` instances. This makes it easy to describe a level without specifying the objects' position structures directly. The levels were selected from puzzles in the official video game. [12]

3.2.3 View

Class diagrams of the main parts of the view are shown in Figures 3.3 and 3.4.

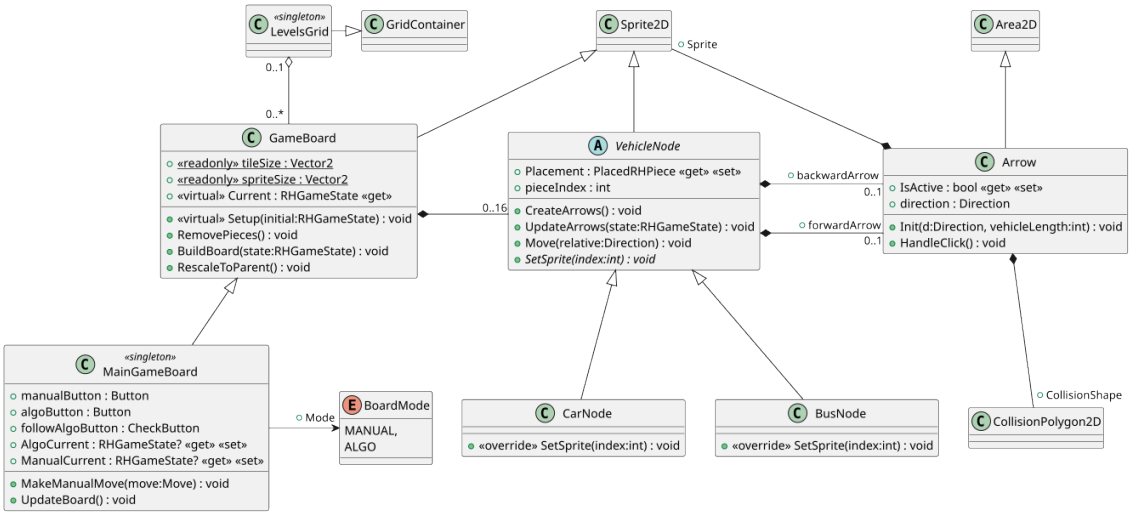


Figure 3.3: Class diagram of the classes used to display the game board

`GameBoard` An object responsible for displaying an `RHGameState`; puzzle previews on the level selection tab use this class. The Game board tab contains an instance of the `MainGameBoard` subclass, which handles switching between Manual and Algorithm modes, displaying the board state, and managing the vehicles' arrows. It subscribes to vertex-click events for manual navigation.

`VehicleNode` / `CarNode` / `BusNode` `Sprite2D` objects representing vehicles. They belong to a `GameBoard` object; the members of this composition can be accessed through Godot's scene-tree parent/child methods. Vehicles belonging to the `MainGameBoard` have `Arrow` objects for manual movement. The arrows are instantiated only once. If a move into the cell indicated by an arrow is not possible, its visibility and interaction are disabled using its `IsActive` field.

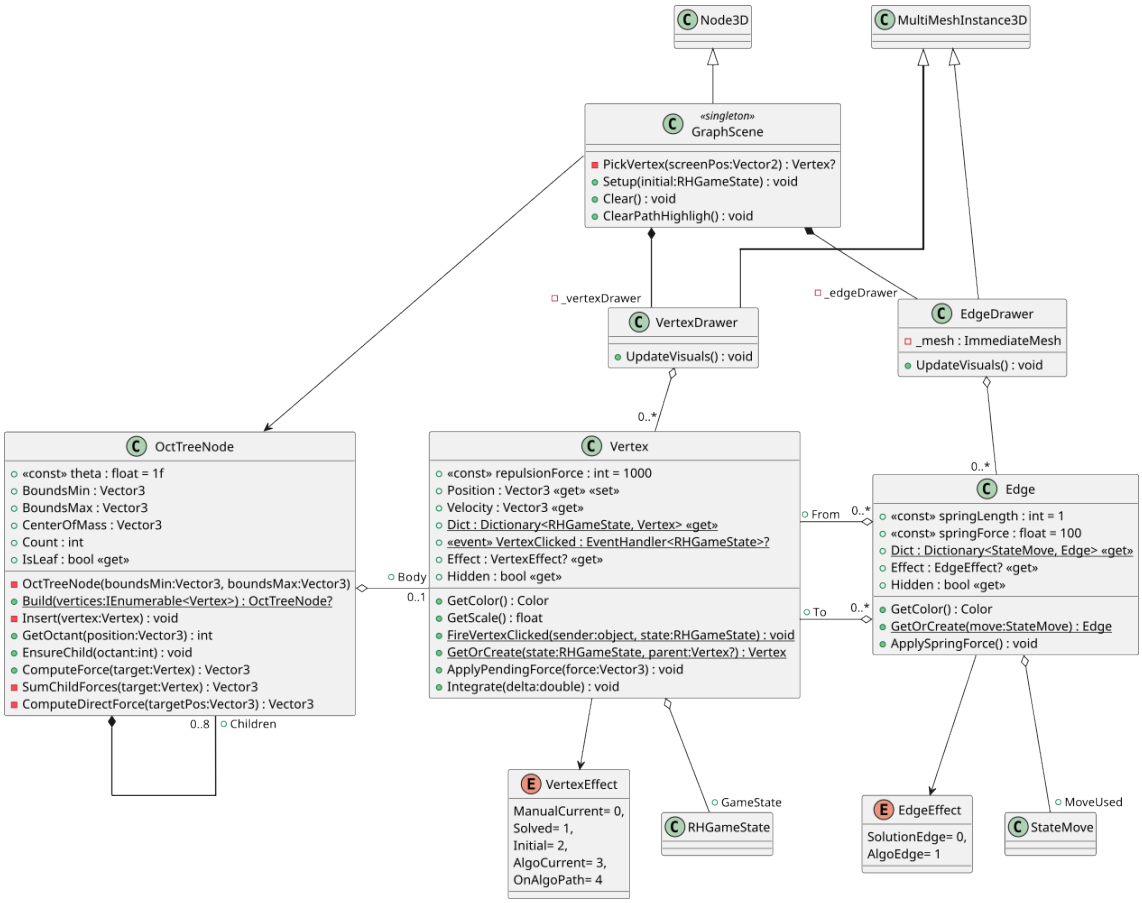


Figure 3.4: Class diagram of the state graph visualization, including the octree implementation adapted to the project

`GraphScene` The 3D root node, which runs the simulation, invokes rendering, and handles slowdowns caused by intensive computation. In each physics frame, it rebuilds the Barnes–Hut octree using the vertices' current positions. It computes the forces for all edges and vertices in parallel, then also computes vertex movement in parallel. To detect vertex clicks, it performs a ray cast based on the camera's current position and orientation.

`Vertex` Represents a state, displayed as a circle in the state graph. It provides a thread-safe data structure for accumulating forces and maintains a priority-based effect system for color and size. It approximates the force acting on it by traversing the constructed octree.

`Edge` Represents a move, displayed as a line in the state graph. The `ApplySpringForce` method adds spring forces to its endpoint vertices. Its color can be queried based on its assigned effect.

`VertexDrawer, EdgeDrawer` Classes responsible for rendering. Instead of giving each vertex and edge its own `Sprite3D` or `ImmediateMesh` object, the components are collected into arrays and used to draw all vertices and edges in batches, significantly speeding up rendering.

`OctTreeNode` Implemented based on the *NetOctree* repository [13], retaining only the methods needed by this project and adding force approximation. It implements the Barnes–Hut algorithm in three-dimensional space.

Other notable classes in the view layer are:

`Camera3d` An orthographic 3D camera belonging to `GraphScene`, with movement, rotation, and zoom controls for inspecting the state graph in detail. It is based on the *3D-RTS-Camera* repository. [14]

`AlgoPlayer` Controls the timing of algorithm execution, subscribes to and unsubscribes from solver events, and handles play, pause, step, and restart operations.

`SolverSettingsTab` Provides the user interface for selecting algorithms and heuristics, setting the tabu size and randomization, and configuring BFS/DFS discovery.

3.2.4 Data Storage

The application does not use a database. Puzzle levels are stored as hard-coded string literals in the `Levels` class. Each level is a seven-line string: the first line contains the level title, and the remaining six lines represent the 6×6 grid. This representation is compact, human-readable, and easy to interpret.

3.2.5 Search Algorithms and Heuristics

This section describes the three search algorithms and the heuristic functions used to guide them.

Heuristic Functions

All four heuristics satisfy three fundamental properties:

- **Nonnegativity**: $h(s) \geq 0$ for every state s.
- **Admissibility**: $h(s) \leq h^*(s)$, where $h^*(s)$ is the minimum number of moves needed to reach a solution from s.
- **Monotonicity** (consistency): for any state s and any successor s' reachable in a single move, $|h(s) - h(s')| \leq 1$.

These properties are required for the graph search to be A^C graph search: when a state is first expanded, the shortest path to it has already been found, so that state never needs to be expanded again. [15, p. 69, Theorem 4.3] [9, p. 16]

Null heuristic. $h(s) = 0$ for every state. This provides no information about the state, turning the search into uninformed brute-force exploration. It is useful for observing the underlying algorithm's behavior.

The following heuristics were implemented specifically for the 6×6 Rush Hour board, with its exit at the right end of the third row.

Distance heuristic. $h(s) = w_{\text{board}} - 1 - x_{\text{main}}$, where w_{board} is the board width and x_{main} is the column containing the front of the main car, indexed from 0. On a board six cells wide, this value ranges from 0 (solved) to 4. It is admissible because the main car must move at least this many cells to reach the exit. It is monotone because a single move changes the main car's distance by at most 1.

Distance + blocking heuristic. Adds a cost of one to the distance heuristic for each cell occupied by a vehicle between the main car and the exit in the exit row. It is admissible because removing each blocking element requires at least one additional move. It is monotone because a single move changes either the distance or the number of blocking cells by at most 1.

Least-moves heuristic. Extends the distance heuristic with a lower bound on the cost of moving blocking vehicles out of the way. For every vehicle blocking the main car's path, it calculates the minimum number of moves required to clear it, taking into account (a) the direction in which it must move—buses must always move downward into rows 4–6 because the exit is in row 3—and (b) other vehicles obstructing that movement. To preserve monotonicity, this analysis cannot be extended in the same way to the number of moves needed to shift the vehicles that block the cars blocking the main car. Using the same estimation method at that level

would make the heuristic nonmonotone: removing such a blocker could decrease the heuristic value by 2.

Tabu Search

Tabu search [16] is a local search metaheuristic. At each step, it discovers all neighboring states, filters out those on the tabu list, evaluates the heuristic for the remaining candidates, and moves to the best remaining neighbor according to that heuristic.

The tabu list stores the last t visited states and requires $O(t)$ memory. The main configurations are:

- $t = 0$: greedy search without memory, always moving to the best neighbor according to the heuristic.
- $t = 1$: a hill-climbing algorithm that only avoids returning to the immediately preceding state.
- $t > 1$: avoids the last t states. It stops without a solution if every neighbor is on the tabu list.

Tabu search does *not* guarantee that a solution will be found or that it will be optimal. It is a fast local method that can become trapped in regions of the state space with favorable heuristic scores but no solution.

Backtracking Search

For each state, backtracking search maintains a sequence of neighboring states that have not yet been visited on the current path, ordered by the heuristic. Its memory requirement is $O(V)$. At each step, it examines the best untried option of the last state on the current path. If no new option remains, it removes that state from the end of the path: the algorithm backtracks.

Termination guarantee: backtracking is guaranteed to terminate on a finite state graph. [15, p. 46, paragraph 3] [17, p. 19] Since the state graph is finite, the search can be regarded as having a depth limit of V. If it terminates without a solution, no solution is reachable from the initial state. However, it avoids only states on the current path, rather than all previously visited states. It may therefore explore large groups of states repeatedly, resulting in substantial redundant work.

Time complexity: in the worst case, the algorithm may explore every simple path from the initial state. For Rush Hour, there is a natural branching bound b such that no state has more than b neighbors. In a space with branching factor b and V states, the algorithm examines $O(b^V)$ paths in the worst case, and the number of steps is also $O(b^V)$. [1, Section 3.4] Under the rules of Rush Hour, the branching factor of any puzzle is at most $2N_{\text{vehicles}}$, where N_{vehicles} is the total number of cars and buses. With a suitable heuristic, the algorithm usually finds a solution much faster, provided one is reachable.

$\mathbf{A^C}$ Graph Search

A^C graph search, also called the consistent algorithm [15, Chapter 4.4], is an informed search algorithm using the evaluation function $f(n) = g(n)+h(n)$ [9, p. 16], where $g(n)$ is the depth and $h(n)$ is the heuristic score. The name A^C follows the notation in Tibor Gregorics's artificial intelligence notes: the "C" refers to the *closed list*, meaning that states that have already been expanded are not expanded again. It is a special variant of the A* algorithm [18] which, with a monotone heuristic, guarantees that the shortest path to a state has already been found when that state is first expanded.

Since all applicable heuristics are nonnegative, admissible, and monotone, the following property holds: when a state is removed from the priority queue for expansion, the path by which it was reached is a shortest path to that state. Each state therefore needs to be expanded at most once. [9, p. 16]

This eliminates the cycle-traversal and rediscovery problems of backtracking. However, the algorithm may still explore many states that are unnecessary for the solution, because it examines every state whose f-value is lower than the cost of the optimal solution.

Time complexity: A^C graph search expands each state at most once, giving at most $O(V)$ expansions. Each expansion generates at most $O(b)$ successors, where b is the branching factor, at most $2 \times N_{\text{vehicles}}$, and each insertion into the priority queue costs $O(\log V)$. The memory requirement is $O(V)$ because of the dictionary of open and expanded states.

The Barnes–Hut Algorithm and the Octree

Force-directed graph layout [4] requires repulsive forces to be calculated between every pair of vertices, costing $O(V^2)$ per frame. With thousands of vertices, this becomes a performance bottleneck.

The Barnes–Hut algorithm [5, 19] reduces this to $O(V \log V)$ using a hierarchical spatial data structure: an **octree** in three dimensions. The algorithm proceeds as follows:

1. **Build the octree**: calculate an axis-aligned cube containing all vertices and use it as the root region, inserting the vertices one by one. Each internal node stores the number of vertices in its subtree and their center of mass. Each leaf contains a single vertex. Building the tree for V vertices takes $O(V \log V)$ time. [5]

2. **Compute the forces**: for each vertex v, invoke the force calculation at the root of the tree. At each node o:

   - If $s/d < \theta$, where s is the side length of the cube represented by o and d is the distance from v to the center of mass of o, treat the entire cell as a single body with a mass equal to the number of vertices it contains. This is the *Barnes–Hut criterion*.
   - Otherwise, recursively traverse the children of o, calculating or approximating each child's contribution to the force on v and applying it to v.

   The procedure takes $O(\log V)$ time per vertex on average. [5] The total force computation for all vertices therefore also takes $O(V \log V)$ time.

3. The parameter θ, typically set within the range 0.5–1.5, controls the tradeoff between accuracy and speed. [5] The application uses $\theta = 1.0$.

The repulsive force between two points follows an inverse-square law: $\vec{F} = -k \cdot m/d^2 \cdot \hat{r}$.

The octree is rebuilt from scratch at every physics update because vertex positions change continuously. Tree construction takes $O(V \log V)$ time, and force calculation takes $O(\log V)$ per vertex, giving a total time complexity of $O(V \log V)$ per update.

3.3 Implementation

3.3.1 Technologies Used

- **Godot Engine 4.6** (Mono/.NET variant) [6]: an open-source game engine providing 2D/3D rendering, physics, and user interface facilities.
- **C# and .NET SDK 9.0** [7]: all application logic was written in C#. It is one of the languages supported by Godot and is well suited to object-oriented logic.

Godot features used:

- `MultiMeshInstance3D` for batched rendering of vertices and edges.
- `Container` objects for constructing the user interface, together with built-in click events for buttons and `CollisionShape2D` objects.
- Execution of the `_PhysicsProcess` method on a background thread to avoid blocking interaction.

3.3.2 Deviations from the Original Design

Force evaluation and sleeping optimization. Initially, each vertex and edge was implemented as a Godot object with its own rendering. Rendering thousands of edges and vertices through separate calls in every frame proved extremely expensive. The following approach was implemented first to address this:

1. Instead of applying forces directly through Godot's physics API, accumulate them in a field on each vertex (`Vector3 _pendingForces`).
2. At the end of each physics update, a `Processor` node iterates over the vertices and calls their evaluation functions.
3. A dynamic threshold determines whether a pending force is significant enough to apply. This threshold scales with the vertex's "age" (`_timeAwake`) and the total number of vertices. Small residual forces in large, stabilized graphs are ignored, while significant forces are applied through Godot's functions.
4. If the forces remain below the threshold, velocity damping eventually brings the vertices to rest. The engine then puts the `RigidBody3D` into "sleep" mode, stopping its movement and calls to `_PhysicsProcess`. A vertex can wake again if an edge's spring force exceeds the threshold and is applied to it.

5. Edges whose endpoints are both asleep skip their line-mesh updates, substantially reducing the rendering load.

This staged optimization (force threshold → physics sleep → skipped rendering) significantly improved performance once the state graph had settled. However, most graphs containing thousands of vertices took too long to settle for this optimization to become useful. The approach therefore had to be redesigned completely: vertices and edges were converted into classes that do not derive from Godot classes, and their rendering was delegated to separate classes. This optimization also made rendering much faster while the graph was actively moving. In addition, when there are thousands of vertices, the physics update rate is reduced from $60/s$ to $10/s$. The application no longer has to wait for one physics update before every UI update; several UI frames can run between physics updates, keeping the interface responsive.

3.4 Testing

3.4.1 Unit Tests

The following unit tests check the correctness of the model layer:

Model Unit Tests		
#	**Test case**	**Verification**
U1	Level parsing	When a known level string is loaded, the level title and the vehicles' count, types (MainCar, Bus, Car), positions, and orientations exactly match the values encoded in the string.
U2	Board grid consistency	After loading, every cell in the internal 6×6 integer matrix of `RHGameState` contains either the appropriate vehicle index or -1; no cell remains uninitialized.
U3	Loading all built-in levels	`Levels.LoadLevels()` successfully loads every built-in level. For each level, verify that its title is nonempty, its first vehicle is a MainCar, its initial state is not solved, and at least one legal move exists.

#	**Test case**	**Verification**
U4	Move generation	`GetPossibleMoves()` and `GetPossibleStateMoves()` consistently return the same number of moves. For every returned move, verify that the affected vehicle remains within the board boundaries in the resulting state and that `StateMove.To` equals the result calculated separately using `WithMove()`.
U5	State immutability	After calling `WithMove()`, the affected vehicle's position in the original state is unchanged; the `==` and `!=` operators identify the new state as different from the original.
U6	State equality	Two states loaded from the same configuration compare equal, with consistent results from `Equals`, `==`, and `!=`. After a move, the states differ. Applying the same move to two separate instances makes them equal again. They also match a state loaded from a separate string describing the modified positions.
U7	Hash consistency	`GetHashCode()` returns the same value for equal states. A different state produced by a move almost certainly has a different value, enabling fast hash-map-based lookup.
U8	Solution detection	`IsSolved()` returns false for a known unsolved state and true for a known solved state. Verify that the exit position is $(5, 2)$.
U9	Null heuristic	Returns 0 for every state.
U10	Distance heuristic range	The value lies in $[0, 4]$ and is 0 only for a solved state.
U11	Distance + blocking heuristic values	Verify that the distance + blocking heuristic returns the expected values for known states.

#	**Test case**	**Verification**
U12	Least-moves heuristic values	Verify that the least-moves heuristic returns the expected values for known states.
U13	Heuristic admissibility	For all four heuristics, $h(s) \leq h^*(s)$, where $h^*(s)$ is the actual minimum number of moves, measured on known puzzles.
U14	Heuristic monotonicity	For all four heuristics, every state s and neighbor s' satisfy $\|h(s) - h(s')\| \leq 1$.
U15	Tabu solver termination	On a solvable puzzle, with a tabu size greater than the number of states, the algorithm either finds a solution or terminates without one.
U16	Backtracking completeness	On a solvable puzzle, backtracking finds a solution with a suitable heuristic.
U17	A^C optimality	For a puzzle with a known optimal solution length, A^C with a monotone heuristic finds a solution of that optimal length.
U18	Backtracking termination on an unsolvable level	On an unsolvable level, backtracking terminates within $O(b^V \cdot V)$ time and reports that no solution exists.
U19	A^C termination on an unsolvable level	On an unsolvable level, graph search terminates within $O(V)$ time and reports that no solution exists.
U20	Solver events are raised	Subscribe to the A^C solver's `NewEdge`, `NewCurrent`, and `Terminated` events. After running the solver, verify that all three events were raised at least once and that `Terminated` was raised with the `Solved` status.
U21	BFS/DFS discovery counts and limits	With a state limit, discovery stops at exactly the specified count. With a sufficiently large limit, it discovers every state.

#	Test case	Verification
U22	StateMove symmetry	A `StateMove` ($A \to B$) equals its reverse ($B \to A$) because edges are undirected. Both equality and hash consistency hold in either direction.

Table 3.2: Model unit tests

View: Vertex and Edge Logic

These tests check the view layer's display logic that is independent of the Godot engine: the effect system, colors, sizes, and physics.

View: Vertex and Edge Logic		
#	**Test case**	**Verification**
V1	Adding, removing, and prioritizing vertex effects	`AddEffect`/`RemoveEffect` correctly update the set of active effects. The `Effect` property always returns the active effect with the smallest enum value, corresponding to the highest priority. After all effects are removed, `Effect` is null.
V2	Vertex color by effect	`GetColor()` returns the documented color for every `VertexEffect` value. A vertex created from a solved state automatically receives the solved effect.
V3	Vertex size by effect	`GetScale()` returns the documented size for every effect value.
V4	Edge effect system and color	On the `Edge` class, `AddEffect`/`RemoveEffect`/`ClearEffects` correctly manage active effects. `GetColor()` returns the correct colors.

#	Test case	Verification
V5	Edge spring force direction	With two vertices placed 2× springLength apart, `ApplySpringForce()` pulls them toward one another; after `Integrate()`, the force's effect is visible in the expected direction. When placed close together (0.5× springLength), the force pushes them apart. This checks the direction and sign of the spring force.
V6	Vertex physics integration	After applying a force to a vertex initially at rest, `Integrate(delta)` changes its velocity and position in the direction of the force. In a second step without force, linear damping reduces its velocity. Applying a large opposing force reverses the velocity.

Table 3.3: Unit tests for view-layer vertex and edge logic

To run the unit tests, execute the following command in the project directory:

```
dotnet add package gdUnit4.api --version 5.1.0-rc1
```

The unit tests can then be run from the editor.

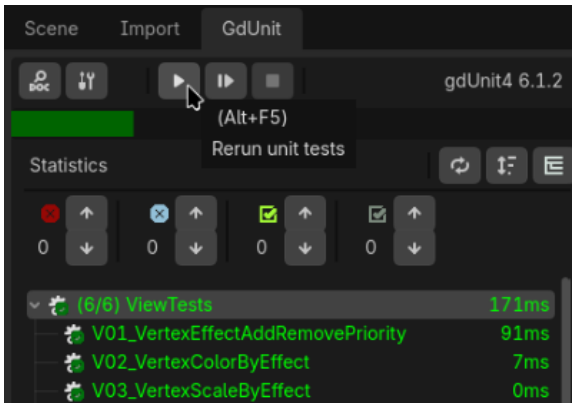


Figure 3.5: Successful execution of the unit tests in the editor

3.4.2 End-User Test Cases

The following test cases are based on the user stories in Table 3.1.

End-User Test Cases				
#	**Action**	**Precondition**	**Input**	**Expected result**
E1	Load a level	The level selection tab is open.	"Level 1" button	The board shows level 1; the graph contains one blue vertex.
E2	Manual move	A level is loaded.	Click a red arrow.	The corresponding car moves in the arrow's direction; a new vertex and edge appear in the graph.
E3	BFS discovery	A level is loaded.	Maximum states: 1000; start BFS.	The graph is populated with at most 1000 vertices; discovered solutions are green.
E4	Pause discovery	Discovery is running.	*Pause* button	Discovery pauses; the vertices settle into position.
E5	Configure the search algorithm	A level is loaded.	Select an algorithm and a heuristic, then press *Apply solver algorithm.*	The algorithm is configured and ready to run.

#	**Action**	**Precondition**	**Input**	**Expected result**
E6	Run the algorithm	An algorithm is configured.	*Start* button	The algorithm begins running.
E7	Adjust algorithm speed	An algorithm is configured.	*Solver algorithm speed* slider	The algorithm executes steps at the selected speed.
E8	Execute one algorithm step	The algorithm has not terminated.	*Step* button	The algorithm executes one step and remains paused.
E9	Restart the algorithm	A level and search algorithm are loaded.	*Restart* button	The algorithm resets; algorithm highlights and the step count return to their initial values.
E10	Switch modes (Manual or Algorithm)	An algorithm is configured.	Manual or Algorithm button	The board switches to the chosen mode, showing either manual moves or the algorithm's steps.
E11	Find a solution	The algorithm is running on a solvable level.	Wait for termination.	Status is Solved; the solution path is green; the solution length is visible.

#	**Action**	**Precondition**	**Input**	**Expected result**
E12	Jump to a discovered state	A level is loaded and some vertices have been discovered.	Left-click a vertex.	The vertex turns red; the board switches to Manual mode and displays that state.
E13	Rotate the camera	The graph is visible.	Right mouse button + mouse movement	The state graph rotates in the direction of the movement.
E14	Zoom	The graph is visible.	Scroll the mouse wheel up/down.	The state graph is displayed closer/farther away.
E15	Move the camera	The graph is visible.	W/A/S/D keys	The camera moves according to the key pressed.
E16	Hide vertices	The graph is visible.	*Hide* toggle	All white vertices and edges are hidden.

Table 3.4: Test cases based on the user stories

Chapter 4

Conclusion

This thesis implemented search algorithms, heuristics, visualization, and interaction within the framework of the Rush Hour logic puzzle. Similar analyses could also be useful for many other problems and games. Three fundamental algorithms were implemented for state-space search, providing a clear illustration of different approaches to a problem. The system can also be used to run a custom algorithm visually, analyze its behavior, and check whether it behaves correctly as expected.

Possible extensions. The application could be developed further in several areas:

- Support for additional puzzle types beyond Rush Hour to generalize the framework.
- Implementation of A* search, with reopening of closed states, for comparison with the A^C algorithm.
- Parallel execution of algorithms to compare different configurations side by side.
- GPU-accelerated force computation to handle graphs containing thousands of vertices.
- Playback of an algorithm's solution on the game board.
- Implementation of a depth limit for backtracking search.
- Restructuring the application according to the MVVM (Model–View–ViewModel) architecture.

Bibliography

[1] Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. 4th. Pearson, 2020. ISBN: 978-0134610993.

[2] ThinkFun Inc. *Rush Hour — Traffic Jam Logic Game*. https://www.thinkfun.com/products/rush-hour/. 1996.

[3] Gary William Flake and Eric B. Baum. "Rush Hour is PSPACE-complete, or "Why you should generously tip parking lot attendants"". In: *Theoretical Computer Science* 270.1–2 (2002), pp. 895–911. DOI: 10.1016/S0304-3975(01)00173-6.

[4] Thomas M. J. Fruchterman and Edward M. Reingold. "Graph Drawing by Force-directed Placement". In: *Software: Practice and Experience* 21.11 (1991), pp. 1129–1164. DOI: 10.1002/spe.4380211102.

[5] Josh Barnes and Piet Hut. "A hierarchical O(N log N) force-calculation algorithm". In: *Nature* 324.6096 (1986), pp. 446–449. DOI: 10.1038/324446a0.

[6] Godot Engine contributors. *Godot Engine – Free and open source 2D and 3D game engine*. https://godotengine.org/. Version 4.6. 2026.

[7] Microsoft. *.NET 9.0 SDK*. https://dotnet.microsoft.com/. 2024.

[8] Glenn E. Krasner and Stephen T. Pope. "A Cookbook for Using the Model-View-Controller User Interface Paradigm in Smalltalk-80". In: *Journal of Object-Oriented Programming* 1.3 (1988), pp. 26–49.

[9] Gregorics Tibor. *Gráfkeresés, heurisztikák tulajdonságai*. https://people.inf.elte.hu/gt/mi/05-06.grafker.pdf.

[10] Robert A. Hearn and Erik D. Demaine. "PSPACE-completeness of sliding-block puzzles and other problems through the nondeterministic constraint logic model of computation". In: *Theoretical Computer Science* 343.1–2 (2005), pp. 72–96. DOI: 10.1016/j.tcs.2005.05.008.

[11] 2swap. *I Solved Klotski*. https://www.youtube.com/watch?v=YGLNyHd2w10. 2025.

[12] ThinkFun. *Rush Hour® Deluxe – The ultimate traffic jam game!* https://store.steampowered.com/app/2073760/Rush_Hour_Deluxe__The_ultimate_traffic_jam_game/. 2022.

[13] Cserép Máté. *A dynamic octree implementation written in C# as a .NET Standard 2.1 library, built on the System.Numerics library.* https://github.com/mcserep/NetOctree/.

[14] Emil Hakala. *A fully customizable and easy-to-use 3D RTS-style Camera for Godot 4.3.* https://github.com/EmilHakala/3D-RTS-Camera.

[15] Fekete István, Gregorics Tibor, and Nagy Sára. *Bevezetés a mesterséges intelligenciába*. Hungarian. Budapest: ELTE Eötvös Kiadó, 2006.

[16] Fred Glover. "Tabu Search—Part I". In: *ORSA Journal on Computing* 1.3 (1989), pp. 190–206. DOI: 10.1287/ijoc.1.3.190.

[17] Gregorics Tibor. *Visszalépéses keresés terminálása*. https://people.inf.elte.hu/gt/mi/04.visszalep.pdf.

[18] Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. "A Formal Basis for the Heuristic Determination of Minimum Cost Paths". In: *IEEE Transactions on Systems Science and Cybernetics* 4.2 (1968), pp. 100–107. DOI: 10.1109/TSSC.1968.300136.

[19] Tom Ventimiglia and Kevin Wayne. *Barnes-Hut Galaxy Simulator*. 2003. URL: https://www.cs.princeton.edu/courses/archive/fall03/cs126/assignments/barnes-hut.html.

List of Figures

2.1 Opening and running the project from the Godot editor 7
2.2 Steps for exporting the project from the Godot editor 8
2.3 The application while an algorithm is running 8
2.4 Manual play and vertex selection 10
2.5 Configuring and applying search algorithms 11
2.6 Controlling algorithm execution 12
2.7 Illustration of the state graph's color coding 14

3.1 Class diagram of the state representation 20
3.2 Class diagram of the search algorithms 21
3.3 Class diagram of the classes used to display the game board 23
3.4 Class diagram of the state graph visualization, including the octree implementation adapted to the project 24
3.5 Successful execution of the unit tests in the editor 35

List of Tables

2.1 Supported platforms and instructions for running the application . . 7
2.2 Color legend for the vertices and edges of the 3D state graph 13

3.1 User stories 19
3.2 Model unit tests 34
3.3 Unit tests for view-layer vertex and edge logic 35
3.4 Test cases based on the user stories 38